

codehive

Python Class Properties

Managing Data within OOP Structures

1. Understanding Properties

Properties are variables that belong to a class or its objects. They are used to store data specific to an entity.

Every object created from a class can have its own unique values for these properties, allowing you to represent different instances of the same type.

The Dot Notation

In Python, we use the **dot (.)** symbol to access or manipulate these properties.

2. Instance vs Class Properties

There are two levels of properties in a Python class:

Instance Properties

Defined inside `__init__`. Unique to each object.

```
self.name = "Emil"
```

Class Properties

Defined outside methods. Shared by all objects.

```
species = "Human"
```

Class properties are great for constants or data that applies to the entire category.

3. Accessing Properties

Accessing data is straightforward using the instance name followed by the property name.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

my_car = Car("Toyota")
print(my_car.brand) # Access via dot notation
```

Tip: If you try to access a property that hasn't been defined, Python will raise an `AttributeError`.

4. Modifying Values

Objects are dynamic. You can change the value of an instance property at any time after the object has been created.

```
p1 = Person("Tobias", 25)
p1.age = 26 # Update the property value
print(p1.age) # 26
```

This is useful for tracking things that change over time, like a user's level in a game or the price of a product.

5. Deleting Properties

You can remove a property from a specific object using the `del` keyword.

```
p1 = Person("Linus", 30)
del p1.age # The 'age' property is now gone
```

Warning: After deletion, attempting to print `p1.age` will result in a crash. However, the class definition remains untouched.

6. Manipulating Class Properties

When you change a **Class Property** using the Class name, the change is reflected across all existing and future objects.

```
class Person:
    lastname = "Default"

p1 = Person("John")
p2 = Person("Jane")

# Changing at the class level
Person.lastname = "Smith"

print(p1.lastname) # Smith
print(p2.lastname) # Smith
```

7. Dynamic Property Addition

Python allows you to add brand-new properties to an object that weren't even in the original class blueprint!

```
p1 = Person("Tobias")  
p1.city = "Oslo" # New property added on the fly  
print(p1.city)
```

Note: This property `city` will only exist for `p1`. Other instances of the `Person` class will not have it.

8. Property Encapsulation Intro

While properties are generally accessible, Python developers use naming conventions to signal privacy:

- `self.name` (Public)
- `self._name` (Protected - do not touch outside the class)
- `self.__name` (Private - names mangled by Python)

Managing how these properties are accessed is the key to building secure software.

9. Practical Exercise

Knowledge Check

Question: How do you access a property of an object?

☐ Using parentheses: `object(property)`

☐ Using brackets: `object[property]`

☒ Using dot notation: `object.property`

10. Mastery Summary

- ✓ Use **Dot Notation** for access/updates.
- ✓ **Instance properties** are for object-specific data.
- ✓ **Class properties** are for shared data.
- ✓ Use **del** to remove attributes.
- ✓ Dynamically add metadata to specific instances if needed.

codehive

Object Data Architecture v2.1